

Лабораторная работа №5

“Нейронные сети. Многослойный перцептрон (MLP)”

Теоретическая часть

Искусственные нейронные сети (ИНС) представляют собой математические модели, вдохновленные работой биологических нейронов. Они состоят из множества соединенных между собой элементов — искусственных нейронов, каждый из которых выполняет простые вычисления, но в совокупности сеть способна решать сложные задачи.

Исторический обзор

- В 1943 году Уоррен Маккалок и Уолтер Питтс предложили первую математическую модель нейрона.
- В 1958 году Фрэнк Розенблат разработал перцептрон — первую нейросеть, способную к обучению.
- В 1986 году Румельхарт, Хинтон и Уильямс предложили алгоритм обратного распространения ошибки, который сделал многослойные сети практически применимыми.
- В 2010-х годах с развитием GPU и больших данных нейронные сети стали основой глубокого обучения.

Архитектура многослойного перцептрона (MLP)

Структура MLP

Многослойный перцептрон (MLP, Multi-Layer Perceptron) — это один из базовых видов искусственных нейронных сетей. Он состоит из нескольких слоев:

- **Входной слой:** получает входные данные (признаки). Количество нейронов = количество признаков.
- **Скрытые слои:** выполняют нелинейное преобразование входных данных.
- **Выходной слой:** формирует предсказания. Количество нейронов зависит от задачи (например, 1 нейрон для бинарной классификации).

Формальное описание нейрона

Каждый искусственный нейрон выполняет следующие вычисления:

1. Линейное преобразование входных данных

$$z = \sum_{i=1}^n w_i x_i + b$$

где:

- X_i — входные значения,
- W_i — веса,
- b — смещение (bias).

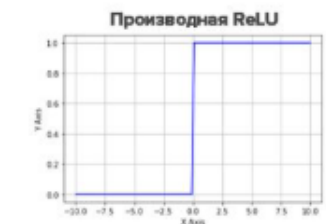
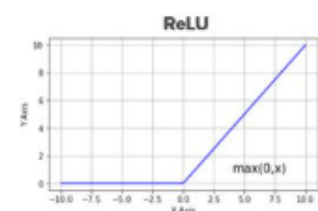
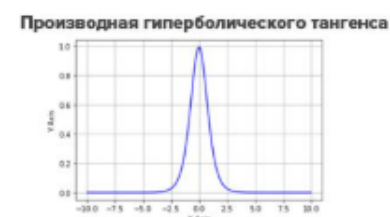
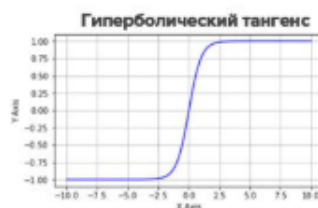
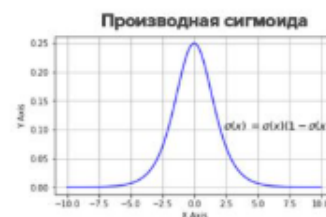
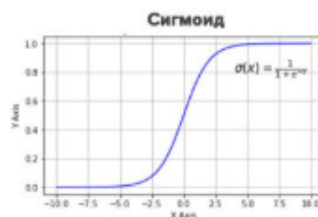
2. Активационная функция

$$a = f(z)$$

где $f(z)$ — функция активации, которая добавляет нелинейность.

Функции активации

Функции активации преобразуют выходное значение нейрона, добавляя нелинейность:



Обучение MLP

Прямой проход (Forward Propagation)

Во время прямого прохода (forward pass) вычисляются выходы каждого слоя:

1. Линейное преобразование:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

2. Активация:

$$a^{(l)} = f(z^{(l)})$$

3. Повторение для всех слоев.

Функция ошибки (Loss function)

Функция ошибки измеряет, насколько предсказания модели отклоняются от истинных значений.

- Для классификации (кросс-энтропия):

$$L = - \sum y_i \log(\hat{y}_i)$$

- Для регрессии (MSE - среднеквадратичная ошибка):

$$L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Обратное распространение ошибки (Backpropagation)

Обратное распространение ошибки (backpropagation) используется для нахождения градиентов и обновления весов.

1. Вычисление градиента ошибки по выходу сети.
2. Вычисление градиентов весов с использованием **цепного правила** производных.

3. Обновление весов с использованием градиентного спуска:

$$W = W - \alpha \frac{\partial L}{\partial W}$$

где α — скорость обучения.

Оптимизаторы

Оптимизаторы улучшают процесс градиентного спуска.

Оптимизатор	Описание
SGD	Стохастический градиентный спуск
Adam	Комбинирует RMSProp и Momentum
RMSProp	Использует адаптивную скорость обучения

Гиперпараметры MLP

Многослойный перцептрон имеет множество параметров, влияющих на его работу.

Гиперпараметр	Описание
Количество слоев	Чем больше слоев, тем сложнее модель
Число нейронов в слоях	Больше нейронов → выше мощность модели
Функция активации	Определяет нелинейность модели
Скорость обучения	Маленькая — долгое обучение, большая — нестабильность
Размер мини-пакета	Влияет на сходимость градиентного спуска
Регуляризация (Dropout, L2)	Предотвращает переобучение

Описание эксперимента

Шаги выполнения лабораторной работы:

1. Загрузка данных.
2. Предобработка данных.
3. Создание и обучение MLP.
4. Оценка качества модели.
5. Сравнение с классическими алгоритмами.

Практическая часть (Пример кода на TensorFlow/Keras)

1. Импорт библиотек и загрузка данных

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

# Генерируем синтетический набор данных (пример - две луны)
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)

# Разделяем на тренировочную и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Нормализация данных
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. Создание многослойного перцептрона (MLP)

```
model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(2,)), # Скрытый слой 1
    keras.layers.Dense(8, activation='relu'), # Скрытый слой 2
    keras.layers.Dense(1, activation='sigmoid') # Выходной слой (бинарная классификация)
])

# Компиляция модели
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])

# Обучение модели
history = model.fit(X_train, y_train, epochs=50, batch_size=16,
                    validation_data=(X_test, y_test))
```

3. Оценка модели

```
# Оцениваем качество на тестовой выборке

y_pred = (model.predict(X_test) > 0.5).astype("int32")

print("Accuracy:", accuracy_score(y_test, y_pred))


# График изменения функции ошибки

plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel('Epochs')

plt.ylabel('Loss')

plt.legend()

plt.show()
```

4. Сравнение с классическими алгоритмами

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.linear_model import LogisticRegression


# Логистическая регрессия

lr = LogisticRegression()

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))


# Случайный лес

rf = RandomForestClassifier(n_estimators=100)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

Задание

“Выборы президента”

Цель работы:

Разработать и обучить нейросеть, которая предсказывает победу на выборах на основе данных об ответах избирателей на 12 вопросов.

Задачи:

1. Подготовить данные для обучения модели.
2. Создать нейросеть с одним скрытым слоем и функцией активации `logsig`.
3. Обучить модель на предоставленных данных.
4. Проверить её точность на тестовых примерах.
5. Визуализировать результаты предсказаний.

Данные:

Входные данные (`dataIn.txt`): матрица ($12 \times N$), где N — количество примеров. Каждый столбец — это 12 ответов одного избирателя.

Выходные данные (`dataOut.txt`): матрица ($2 \times N$), где каждая колонка `[1 0]` означает победу правящей партии, а `[0 1]` — победу оппозиции.

Данные:

```
# Генерация тестовых данных

np.random.seed(42)

X = np.random.randint(0, 2, size=(100, 12)) # 100 примеров, 12 бинарных признаков
Y = np.random.randint(0, 2, size=(100, 2)) # 100 примеров, 2 класса (one-hot encoding)

# Сохранение данных в файлы

np.savetxt('dataIn.txt', X, fmt='%d')
np.savetxt('dataOut.txt', Y, fmt='%d')
```